

MDISC (Matemática Discreta)

Autores

Diogo Moreira - 1210527
Susana Loureiro - 1220804
Francisco Oliveira-1230684
Inês Oliveira-1231205

Professor

AIM
ALG
ACD
SAU

Disciplina

MDISC

Conteúdo

Explicação dos procedimentos 3

Explicação dos procedimentos

```
public static void sortArrayListPrimitivePerPrice(ArrayList<Edge> edges) {  
    Edge saveEdge;  
    for (int i = 0; i < edges.size() - 1; i++) {  
        for (int j = i + 1; j < edges.size(); j++) {  
            if (edges.get(j).getPrice() < edges.get(i).getPrice()) {  
                saveEdge = edges.get(i);  
                edges.set(i, edges.get(j));  
                edges.set(j, saveEdge);  
            }  
        }  
    }  
}
```

FIGURA 1 - SORT ARRAY LIST PER PRICE

Este método tem como função ordenar as arestas pelo custo.

```
public static ArrayList<Point> numberOfVertices(ArrayList<Edge> edges) {  
    ArrayList<Point> vertices = new ArrayList<>();  
    boolean value1;  
    boolean value2;  
    for (Edge edgeX : edges) {  
        value1 = false;  
        value2 = false;  
        Point x1 = edgeX.getP1();  
        Point x2 = edgeX.getP2();  
        for (Point pointX : vertices) {  
            if (x1.equals(pointX)) {  
                value1 = true;  
            }  
            if (x2.equals(pointX)) {  
                value2 = true;  
            }  
        }  
        if (!value1) {  
            vertices.add(x1);  
        }  
        if (!value2) {  
            vertices.add(x2);  
        }  
    }  
    return vertices;  
}
```

FIGURA 2 - NUMBER OF VERTICES

Este método tem como função determinar os vértices do grafo, para depois determinar onde o algoritmo de Kruskal deve parar.

```
private static void distributeInArrayPoints(Point[][] s, ArrayList<Point> vertices)
{
    int i = 0;
    for (Point vertice : vertices) {
        s[0][i] = vertice;
        i++;
    }
}
```

FIGURA 3 - DISTRIBUTE IN ARRAY POINTS

Este método tem como função distribuir os vértices pela primeira linha.

```
public static int findNull(int column, Point[][] S) {
    for (int i = 0; i < S[column].length; i++) {
        if (S[i][column] == null) {
            return i;
        }
    }
    return -1;
}
```

FIGURA 4 - FIND NULL

Este método tem como função encontrar uma linha, coluna que esteja vazia para colocar o vértice a ser agrupado.

```
public static ArrayList<Edge> kruskalAlgorithm(ArrayList<Edge> edges) {
    int na = 0; //n° de arestas do grafo resultante
    ArrayList<Point> vertices = numberOfVertices(edges); //vertices do grafo
    ArrayList<Edge> edgesSave = new ArrayList<>(); //arestas que vão ser incluídas no grafo resultante
    Point[][] S = new Point[vertices.size()][vertices.size()]; //a tabela do algoritmo (aquela parte do S1, S2, (...))
    distributeInArrayPoints(S, vertices); //preenche a primeira linha da tabela com cada vértice

    //x1 e x2 são os vértices de cada aresta analisada para ser incluída no grafo resultante
    int valueIndexLine1 = 0; //índice da linha do vértice x1
    int valueIndexLine2 = 0; //índice da linha do vértice x2
    int valueIndexColumn1 = 0; //índice da coluna do vértice x1
    int valueIndexColumn2 = 0; //índice da coluna do vértice x2
    for (Edge edgeX : edges) {
        Point x1 = edgeX.getP1();
        Point x2 = edgeX.getP2();
        for (int j = 0; j < S[0].length; j++) {
            for (int k = 0; k < S.length; k++) {
                //determina onde estão cada um dos vértices na tabela
                if (x1.equals(S[k][j])) {
                    valueIndexColumn1 = j;
                    valueIndexLine1 = k;
                }
                if (x2.equals(S[k][j])) {
                    valueIndexColumn2 = j;
                    valueIndexLine2 = k;
                }
            }
        }
    }
}
```

FIGURA 5 - KRUSKAL ALGORITHM

```

    if (valueIndexColumn1 != valueIndexColumn2) { //se estiverem em colunas diferentes, então a aresta vai ser adicionada
        na++;
        if (na == vertices.size()) { //o algoritmo pára quando o número de arestas no grafo resultante é =n° de vértices-1
            break;
        }
        edgesSave.add(edgeX);
        //coloca o vértice que precisa de ser agrupado numa das linhas da coluna onde está o outro vértice
        int newIndexLine2 = findNull(valueIndexColumn1, S);
        for (int l = 0; l < S[valueIndexColumn2].length; l++) {
            if (S[l][valueIndexColumn2] == null) {
                break;
            }
            S[newIndexLine2][valueIndexColumn1] = S[l][valueIndexColumn2];
            S[l][valueIndexColumn2] = null;
            newIndexLine2++;
        }
    }
}
return edgesSave;
}

```

FIGURA 6 - KRUSKAL ALGORITHM CONTINUATION

Este método é o algoritmo Kruskal, com a sua explicação comentada no código. Ficou dividido em duas imagens para melhor entendimento, visto que o algoritmo é demasiado grande para colocar em uma só imagem.